Лабораторна робота 8. Кошик, спільний стан і база даних у проєкті Blazor

Завдання: продовження власного застосунку

Продовжіть проєкт із лабораторної 7. **Новий проєкт не створюйте.** Збережіть власну тематику, моделі, формулу, зображення та оформлення.

Замість одного підтвердженого результату реалізуйте кошик із кількома позиціями. Перенесіть каталог до SQLite, організуйте читання через сервіс та перевірте поведінку стану під час навігації й перезапуску.

Обов'язковий результат

1. Кошик додає, видаляє та очищує позиції; показує кількість і суму у гривнях.
2. Компоненти отримують один scoped-сервіс кошика через DI й реагують на його подію.
3. Каталог із щонайменше шести власних товарів читається з SQLite через EF Core та окремий сервіс.
4. Є стани завантаження, порожнього результату й помилки.
5. Спільні назва й валюта передаються каскадним параметром.
6. Друга сторінка дозволяє перевірити збереження кошика під час інтерактивних переходів.

Оплата, автентифікація та збереження оформлених замовлень не входять до завдання. У БД зберігається **тільки каталог**. Порівняння з HTTP-клієнтом наприкінці — необов'язкове.

Мета: розрізняти локальний стан компонента, спільний стан сесії та постійні дані; використовувати DI, події сервісу й короткоживучі контексти EF Core.

Перед початком перевірте `dotnet build` і збережіть резервну копію. Початкова версія має `Drink`, `DrinkDraft`, `OrderLine`, `MoneyLabel`, `DrinkCard`, `ConfigureDrinkDialog` та Home із шістьма товарами і полем `lastSelection`. Сервісів, контексту БД та `ShopSettings` ще немає.

Готове демо можна використовувати для порівняння, але наступні лістинги «Було» відповідають саме результату лабораторної 7. Власні перейменовані типи й опції зберігайте під час замін.

Як читати лістинги зі змінами

Формат лістинга залежить від обсягу роботи:

- **Новий файл** — повний вміст у блоці `csharp` або `razor`.
- **Кілька локальних змін** — короткий `diff` із контекстом.
- **Значна перебудова** — спочатку повний старий вміст («Було»), нижче повний новий («Стало»).
- **Оформлення CSS** — лише новий повний вміст файла.

У блоці `diff` позначення читаються так:

- `-` на початку рядка — вилучіть цей рядок.
- `+` на початку рядка — додайте цей рядок без початкового знака `+`.
- Пробіл на початку — незмінний контекст, який допомагає знайти потрібне місце.
- `--- a/...` і `+++ b/...` — назва файла до та після зміни; це не рядки програми.
- `@@ ... @@` — межі фрагмента і номери рядків; їх не потрібно копіювати у файл.

Не замінюйте весь файл одним фрагментом diff. Застосовуйте зміни послідовно: кожен diff порівнює код із попереднім етапом цієї роботи, а не завжди з початковим шаблоном. Пропущені між фрагментами рядки залишаються без змін. Пробіл контексту та службові позначення diff не належать до коду; відступи самого коду потрібно зберегти.

У різних виправленнях SDK можуть відрізнятися коментарі й форматування шаблону. Орієнтуйтеся на назви файлів і навколишній код, а не лише на номери рядків.

1. Спільний стан кошика

Стан — дані, які можуть змінюватися під час роботи інтерфейсу. **Сервіс** — звичайний клас із певною відповідальністю. Передавання сервісів через DI дозволяє компонентам отримувати спільний екземпляр, не створюючи його через `new` у кожному компоненті.

Що залишити спільним. Контейнер кошика не повинен знати, що таке молоко, рамка чи пакування. Він додає й видаляє готові позиції та підсумовує `Total`. Тут переважно зберігається загальна логіка демонстрації; тематичні обчислення належать чернетці.

Створіть папку **Services** та файл **OrderState.cs**:

```
using CampusCafe.Models;

namespace CampusCafe.Services;

public class OrderState
{
    private readonly List<OrderLine> items = new();
    public IReadOnlyList<OrderLine> Items => items.AsReadOnly();
    public decimal Total => items.Sum(item => item.Total);
    public int Count => items.Sum(item => item.Quantity);
    public event Action? Changed;

    public void Add(OrderLine item)
    {
        if (item.Quantity is < 1 or > 10 || item.UnitPrice < 0)
            throw new ArgumentException("Некоректна позиція замовлення.");
        items.Add(item);
        Changed?.Invoke();
    }

    public void Remove(Guid id)
    {
        items.RemoveAll(item => item.Id == id);
        Changed?.Invoke();
    }
```

```
    public void Clear()
    {
        items.Clear();
        Changed?.Invoke();
    }
}
```

`items` прихований від зовнішнього коду. `Items` дає доступ лише для читання, а зміни виконуються методами `Add`, `Remove` та `Clear`. Тому всі зміни проходять через одне місце, яке повідомляє інтерфейс.

`Count` рахує кількість порцій, а `Items.Count` — кількість рядків. Один рядок із двома порціями має `Count = 2`, але `Items.Count = 1`.

`Changed` — подія C#, не HTML-подія. `Changed?.Invoke()` викликає підписані обробники, якщо вони є. Саме повідомлення не перемальовує сторінку: компонент-підписник має попросити Blazor про оновлення.

У Interactive Server сервіс із `AddScoped<OrderState>()` спільний у межах **circuit** — інтерактивної сесії користувача. Це не загальний кошик усіх відвідувачів і не постійне сховище. Повернення між інтерактивними сторінками зберігає цей стан, але повне перезавантаження в цьому застосунку створює нову сесію.

2. Реєстрація сервісу та компонент кошика

У **Program.cs** внесіть дві локальні зміни:

```
--- a/Program.cs
+++ b/Program.cs
@@ -1,10 +1,13 @@
 using CampusCafe.Components;
+using CampusCafe.Services;

 var builder = WebApplication.CreateBuilder(args);

 // Add services to the container.
 builder.Services.AddRazorComponents()
     .AddInteractiveServerComponents();
+
+builder.Services.AddScoped<OrderState>();

 var app = builder.Build();
```

У **Components/_Imports.razor** додайте імпорт сервісів:

```
 @using CampusCafe.Models
 @using CampusCafe.Components.Shared
+@using CampusCafe.Services
```

Реєстрація `AddScoped` дозволяє DI створювати екземпляр і надавати його компонентам однієї інтерактивної сесії. Не використовуйте `AddSingleton` для персонального кошика: він був би спільним для різних користувачів.

Адаптуйте зараз — опис підтвердженої позиції. Змініть `item.DrinkName`, `item.Volume`, `item.Milk` та одиниці вимірювання на відповідні властивості своєї позиції. Кошик повинен показувати опції, які справді обрано в діалозі. Видалення за `item.Id`, кількість, суму й підписку на `Changed` залиште за тією самою логікою.

Створіть **Components/Shared/CartSummary.razor**:

```
@implements IDisposable
@inject OrderState Order

<aside class="cart" aria-label="Кошик">
    <span class="eyebrow">Ваша перерва</span>
    <h2>Кошик <span class="count">@Order.Count</span></h2>
    @if (Order.Items.Count == 0)
    {
        <p class="empty-cart">Оберіть напій у каталозі.<br />Він з’явиться тут після
підтвердження.</p>
    }
    else
    {
        <ul class="cart-items">
            @foreach (var item in Order.Items)
            {
                <li @key="item.Id">
                    <strong>@item.DrinkName</strong>
                    <p>@item.Volume мл · @item.Milk молоко · @item.Quantity шт.</p>
                    <div class="cart-line-footer">
                        <MoneyLabel Amount="item.Total" />
                        <button type="button" class="text-button" @onclick="() =>
Order.Remove(item.Id)"
                                aria-label="@($"Видалити
{item.DrinkName}")">Видалити</button>
                    </div>
                </li>
            }
        </ul>
        <div class="cart-total" aria-live="polite">
            <span>До сплати</span><strong><MoneyLabel Amount="Order.Total" /></strong>
        </div>
        <button type="button" class="secondary full-width" @onclick="Order.Clear">Очистити
кошик</button>
    }
    <p class="cart-note">Навчальний кошик. Оплата та збереження замовлень не виконуються.
</p>
</aside>

@code {
    protected override void OnInitialized() => Order.Changed += Refresh;
    private void Refresh() => _ = InvokeAsync(StateHasChanged);
    public void Dispose() => Order.Changed -= Refresh;
}
```

`@inject OrderState Order` отримує контейнер стану з DI. Не замінюйте це на `new OrderState()`: тоді кошик одержить інший, порожній екземпляр.

Під час ініціалізації компонент підписується на `Changed`. Коли інший компонент додає позицію, викликається `Refresh`, а `InvokeAsync(StateHasChanged)` планує оновлення в контексті рендерера Blazor. `_ =` тут відкидає повернений `Task`, бо подія `Action` синхронна; обробник лише ставить оновлення в чергу.

`IDisposable` і `Dispose()` потрібні для відписки, коли компонент прибирається з дерева, наприклад під час переходу на іншу сторінку. Інакше сервіс може тримати посилання на старий компонент.

`@key="item.Id"` допомагає Blazor зіставляти рядки з об'єктами під час видалення. Видаляємо за ідентифікатором позиції, а не за назвою напою: у кошику можуть бути два різні капучино.

3. Від одного результату до кошика

Спочатку перевіримо кошик без зміни джерела каталогу. Тому список із шести товарів поки залишається у Home. Замість `lastSelection = line` викликаємо `Order.Add(line)`; компонент `CartSummary` показує накопичені позиції.

Components/Pages/Home.razor — було:

```
@page "/"
<PageTitle>Каталог · Campus Café</PageTitle>

<section class="hero">
    <p class="eyebrow">LNTU · кава між парами</p>
    <h1>Campus Café</h1>
    <p>Оберіть напій і налаштуйте його. Поки що зберігаємо лише останній вибір.</p>
</section>

<div class="catalog">
    @foreach (var drink in drinks)
    {
        <DrinkCard @key="drink.Id" Drink="drink" OnSelect="SelectDrink" />
    }
</div>

<section aria-live="polite">
    <h2>Останній підтверджений вибір</h2>
    @if (lastSelection is null)
    {
        <p>Ви ще нічого не підтвердили.</p>
    }
    else
    {
        <p>@lastSelection.DrinkName · @lastSelection.Volume мл · @lastSelection.Milk молоко
· @lastSelection.Quantity шт.</p>
        <strong><MoneyLabel Amount="lastSelection.Total" /></strong>
    }
</section>

@if (selectedDrink is not null)
```

```
{
    <ConfigureDrinkDialog @key="selectedDrink.Id" Drink="selectedDrink"
                          OnCancel="CloseDialog" OnConfirm="ConfirmDrink" />
}

@code {
    private Drink? selectedDrink;
    private OrderLine? lastSelection;
    private readonly List<Drink> drinks = new()
    {
        new() { Id = 1, Name = "Капучино", Description = "Еспресо, молоко та ніжна пінка.",
BasePrice = 65m },
        new() { Id = 2, Name = "Латте", Description = "М’який смак і більше молока.",
BasePrice = 70m },
        new() { Id = 3, Name = "Мокачино", Description = "Кава з шоколадом.", BasePrice =
85m },
        new() { Id = 4, Name = "Матча-латте", Description = "Зелений чай матча з молоком.",
BasePrice = 90m },
        new() { Id = 5, Name = "Какао", Description = "Шоколадний напій без кави.",
BasePrice = 60m },
        new() { Id = 6, Name = "Чай-латте", Description = "Пряний чай із молоком.",
BasePrice = 75m }
    };

    private void SelectDrink(Drink drink) => selectedDrink = drink;
    private void CloseDialog() => selectedDrink = null;
    private void ConfirmDrink(OrderLine line)
    {
        lastSelection = line;
        CloseDialog();
    }
}
```

Стало — повний файл:

```
@page "/"
@inject OrderState Order
<PageTitle>Каталог · Campus Café</PageTitle>

<section class="hero">
    <p class="eyebrow">LNTU · кава між парами</p>
    <h1>Campus Café</h1>
    <p>Оберіть напій і налаштуйте його. Підтверджені позиції додаються до кошика.</p>
</section>

<div class="shop-grid">
    <section>
<div class="catalog">
    @foreach (var drink in drinks)
    {
        <DrinkCard @key="drink.Id" Drink="drink" OnSelect="SelectDrink" />
    }
</div>
    </section>
    <CartSummary />
</div>
```

```
@if (selectedDrink is not null)
{
    <ConfigureDrinkDialog @key="selectedDrink.Id" Drink="selectedDrink"
                          OnCancel="CloseDialog" OnConfirm="ConfirmDrink" />
}

@code {
    private Drink? selectedDrink;
    private readonly List<Drink> drinks = new()
    {
        new() { Id = 1, Name = "Капучино", Description = "Еспресо, молоко та ніжна пінка.",
BasePrice = 65m },
        new() { Id = 2, Name = "Латте", Description = "М’який смак і більше молока.",
BasePrice = 70m },
        new() { Id = 3, Name = "Мокачино", Description = "Кава з шоколадом.", BasePrice =
85m },
        new() { Id = 4, Name = "Матча-латте", Description = "Зелений чай матча з молоком.",
BasePrice = 90m },
        new() { Id = 5, Name = "Какао", Description = "Шоколадний напій без кави.",
BasePrice = 60m },
        new() { Id = 6, Name = "Чай-латте", Description = "Пряний чай із молоком.",
BasePrice = 75m }
    };

    private void SelectDrink(Drink drink) => selectedDrink = drink;
    private void CloseDialog() => selectedDrink = null;
    private void ConfirmDrink(OrderLine line)
    {
        Order.Add(line);
        CloseDialog();
    }
}
```

У **Components/Shared/ConfigureDrinkDialog.razor** змініть лише напис кнопки:

```
-           <button type="button" @onclick="ConfirmAsync">Підтвердити вибір</button>
+           <button type="button" @onclick="ConfirmAsync">Додати до кошика</button>
```

Контрольна точка: додайте капучино за 200 грн і латте за 70 грн: два рядки, три порції, 270 грн. Видаліть лише капучино, потім очистіть кошик. Скасування нового діалогу не повинно змінювати кошик. База даних для цих дій ще не потрібна.

4. SQLite та Entity Framework Core

SQLite — база даних у файлі. **Entity Framework Core** — бібліотека, яка зіставляє C#-об’єкти з таблицями та виконує SQL-запити. **DbContext** — об’єкт сеансу роботи з базою.

Зупиніть сервер та додайте пакет:

```
dotnet add package Microsoft.EntityFrameworkCore.Sqlite --version 10.0.12
```

У цій роботі версію зафіксовано для відтворюваності. Не змішуйте основні версії пакетів EF Core. Після встановлення в `.csproj` буде `PackageReference`, а при `dotnet restore` NuGet завантажить бібліотеки.

Створіть папку **Data** і файл **CafeDbContext.cs**:

```
using CampusCafe.Models;
using Microsoft.EntityFrameworkCore;

namespace CampusCafe.Data;

public class CafeDbContext : DbContext
{
    public CafeDbContext(DbContextOptions<CafeDbContext> options) : base(options) { }

    public DbSet<Drink> Drinks => Set<Drink>();
}
```

`DbSet<Drink>` представляє набір напоїв. `DbContextOptions` передає налаштування підключення до базового класу. EF Core розпізнає властивість `Id` як первинний ключ за домовленістю.

Адаптуйте зараз — каталог у базі. Перед першим запуском замініть усі шість об'єктів у `AddRange`: назви, описи, базові ціни й ілюстрації мають належати вашій темі. `new Drink` замініть на свій тип, якщо ви його перейменували. Не залишайте в базі кавовий каталог, плануючи змінити лише заголовки карток.

Якщо перейменували модель, узгодьте також `DbSet<Drink>` / `Drinks`, звернення `db.Drinks` у seed і запит у `CatalogService`. Зміну назви таблиці або полів після створення SQLite потрібно розглядати як зміну схеми, а не як зміну оформлення.

Створіть **Data/SeedData.cs**:

```
using CampusCafe.Models;
using Microsoft.EntityFrameworkCore;

namespace CampusCafe.Data;

public static class SeedData
{
    public static async Task InitializeAsync(IDbContextFactory<CafeDbContext> factory)
    {
        await using var db = await factory.CreateDbContextAsync();
        await db.Database.EnsureCreatedAsync();
        if (await db.Drinks.AnyAsync()) return;

        db.Drinks.AddRange(
            new Drink { Name = "Капучино", Description = "Еспресо, молоко та ніжна пінка.",
Icon = "☕", BasePrice = 65m },
            new Drink { Name = "Латте", Description = "М'який смак і більше молока.", Icon =
"🥛", BasePrice = 70m },
            new Drink { Name = "Мокачино", Description = "Кава з шоколадом для солодкої
паузи.", Icon = "🍫", BasePrice = 85m },
            new Drink { Name = "Матча-латте", Description = "Зелений чай матча з молоком.",
```

```
Icon = "☕", BasePrice = 90m },
            new Drink { Name = "Какао", Description = "Теплий шоколадний напій без кави.",
Icon = "🤎", BasePrice = 60m },
            new Drink { Name = "Чай-латте", Description = "Пряний чай із молоком і
корицею.", Icon = "🍵", BasePrice = 75m }
        );
        await db.SaveChangesAsync();
    }
}
```

Початкове наповнення, або seed, створює навчальні записи. `EnsureCreatedAsync()` створює відсутню базу й схему. `AnyAsync()` перевіряє наявність товарів, тому повторний запуск не додає ще шість дублікатів.

Зміна тексту `SeedData.cs` не змінює наявні записи автоматично. Для подальшого розвитку потрібні операції оновлення, а для зміни схеми — міграції. `EnsureCreated` не є заміною міграцій. Для повторного чистого досліду можна, зупинивши застосунок, **перемістити навчальний `cafe.db` у резервну папку**, не видаляючи цінні дані, і запустити застосунок знову.

У цій роботі база містить лише каталог. `OrderLine` не включено до `DbSet`, тому додавання в кошик не викликає `SaveChangesAsync()`.

Перевірте: де зараз зберігаються ваші шість товарів і що станеться під час другого запуску seed? Поясніть роль `AnyAsync()` перед тим, як запускати застосунок повторно.

5. Сервіс читання каталогу

Створіть **Services/CatalogService.cs**:

```
using CampusCafe.Data;
using CampusCafe.Models;
using Microsoft.EntityFrameworkCore;

namespace CampusCafe.Services;

public class CatalogService
{
    private readonly IDbContextFactory<CafeDbContext> factory;

    public CatalogService(IDbContextFactory<CafeDbContext> factory)
    {
        this.factory = factory;
    }

    public async Task<List<Drink>> GetDrinksAsync()
    {
        await using var db = await factory.CreateDbContextAsync();
        return await db.Drinks.AsNoTracking().OrderBy(d => d.Name).ToListAsync();
    }
}
```

`IDbContextFactory<CafeDbContext>` створює новий контекст для кожної операції. Це корисно для серверного Blazor: компоненти живуть довго, а один `DbContext` не призначений для одночасної роботи кількох операцій.

`await using` звільняє контекст після завершення методу. `AsNoTracking()` вимикає відстеження змін для каталогу, який ми лише читаємо. `OrderBy` сортує за назвою. `ToListAsync()` виконує запит та повертає готовий список.

`Task<List<Drink>>` означає асинхронну операцію, результатом якої буде список. `await` очікує її завершення без синхронного блокування потоку на весь час очікування; він не означає автоматичного запуску нового потоку.

Тепер структура розширилася:

```
CampusCafe/
├── Models/          Drink, DrinkDraft, OrderLine
├── Data/            CafeDbContext, SeedData
├── Services/        CatalogService, OrderState
├── Components/      компоненти лабораторної 7 та Home з кошиком
└── Program.cs       наступний крок — реєстрація створених класів
```

6. Підключення БД у Program.cs і перевірка каталогу

У **Program.cs** змінюється кілька частин: імпорти, реєстрація сервісів, маршрут API та початкове наповнення бази. Порівняйте повні версії файла. У блоці «Було» вже є реєстрація кошика з цієї роботи. Пояснювальні коментарі у власному коді можна зберегти.

Було — вміст файла на попередньому етапі:

```
using CampusCafe.Components;
using CampusCafe.Services;

var builder = WebApplication.CreateBuilder(args);

// Add services to the container.
builder.Services.AddRazorComponents()
    .AddInteractiveServerComponents();

builder.Services.AddScoped<OrderState>();

var app = builder.Build();

// Configure the HTTP request pipeline.
if (!app.Environment.IsDevelopment())
{
    app.UseExceptionHandler("/Error", createScopeForErrors: true);
}
app.UseStatusCodePagesWithReExecute("/not-found", createScopeForStatusCodePages: true);
app.UseAntiforgery();

app.MapStaticAssets();
app.MapRazorComponents<App>()
```

```
    .AddInteractiveServerRenderMode();

app.Run();
```

Стало — повний вміст після заміни:

```
using CampusCafe.Components;
using CampusCafe.Data;
using CampusCafe.Services;
using Microsoft.EntityFrameworkCore;

var builder = WebApplication.CreateBuilder(args);

builder.Services.AddRazorComponents()
    .AddInteractiveServerComponents();

var databasePath = Path.Combine(builder.Environment.ContentRootPath, "cafe.db");
builder.Services.AddDbContextFactory<CafeDbContext>(options =>
    options.UseSqlite($"Data Source={databasePath}"));
builder.Services.AddScoped<CatalogService>();
builder.Services.AddScoped<OrderState>();

var app = builder.Build();

if (!app.Environment.IsDevelopment())
{
    app.UseExceptionHandler("/Error", createScopeForErrors: true);
}
app.UseStatusCodePagesWithReExecute("/not-found", createScopeForStatusCodePages: true);
app.UseAntiforgery();

app.MapStaticAssets();
app.MapGet("/api/drinks", async (CatalogService catalog) =>
    await catalog.GetDrinksAsync());
app.MapRazorComponents<App>()
    .AddInteractiveServerRenderMode();

await SeedData.InitializeAsync(
    app.Services.GetRequiredService<IDbContextFactory<CafeDbContext>>());

app.Run();
```

`ContentRootPath` — папка вмісту застосунку. `Path.Combine` формує шлях до `cafe.db`. `UseSqlite` вибирає провайдер; без пакета або `using Microsoft.EntityFrameworkCore` цей метод не буде доступний.

`AddScoped<CatalogService>()` і `AddScoped<OrderState>()` роблять класи доступними через DI. Реєстрація — ще не виклик їхніх методів.

Початкове наповнення виконується **до** `app.Run()`, щоб запити не отримали порожню, ще не підготовлену базу.

`MapGet("/api/drinks", ...)` створює кінцеву точку Minimal API. Відкриття цієї адреси в браузері надсилає GET-запит; список об'єктів автоматично серіалізується у JSON. Клас контролера для

такого endpoint не обов’язковий.

Запустіть:

```
dotnet run --urls http://localhost:5127
```

Відкрийте `http://localhost:5127/api/drinks`. **Контроль:** масив із шести напоїв із полями `id`, `name`, `description`, `icon`, `basePrice`. Поряд із `.csproj` з’явиться `cafe.db`. Головна сторінка ще читає локальний список із шести товарів. У наступному кроці замінимо лише цей шлях отримання даних та пов’язану розмітку станів.

7. Завантаження даних у Home

Адаптуйте зараз — збирання власної сторінки. Перенесіть вже обрані назви типів, сервісів і компонентів до нового лістинга. Змініть `PageTitle`, головний заголовок, рекламний текст, повідомлення помилок і назву секції каталогу. Дані тепер надходять із вашого seed через сервіс, тому локальний список із шести товарів треба прибрати.

Перед зміною знайдіть три зв’язки: `OnSelect → SelectDrink`, `OnConfirm → ConfirmDrink` і `Order.Add → Changed`. Назви методів у власному проєкті можуть бути іншими, але напрямок передавання даних має залишитися зрозумілим.

У **Components/Pages/Home.razor** перейдіть від каталогу в пам’яті з розділу 3 до каталогу з БД:

Було — вміст файла на попередньому етапі:

```
@page "/"
@inject OrderState Order
<PageTitle>Каталог · Campus Café</PageTitle>

<section class="hero">
    <p class="eyebrow">LNTU · кава між парами</p>
    <h1>Campus Café</h1>
    <p>Оберіть напій і налаштуйте його. Підтверджені позиції додаються до кошика.</p>
</section>

<div class="shop-grid">
    <section>
<div class="catalog">
    @foreach (var drink in drinks)
    {
        <DrinkCard @key="drink.Id" Drink="drink" OnSelect="SelectDrink" />
    }
</div>
    </section>
    <CartSummary />
</div>

@if (selectedDrink is not null)
{
    <ConfigureDrinkDialog @key="selectedDrink.Id" Drink="selectedDrink"
                          OnCancel="CloseDialog" OnConfirm="ConfirmDrink" />
```

```
}

@code {
    private Drink? selectedDrink;
    private readonly List<Drink> drinks = new()
    {
        new() { Id = 1, Name = "Капучино", Description = "Еспресо, молоко та ніжна пінка.",
BasePrice = 65m },
        new() { Id = 2, Name = "Латте", Description = "М’який смак і більше молока.",
BasePrice = 70m },
        new() { Id = 3, Name = "Мокачино", Description = "Кава з шоколадом.", BasePrice =
85m },
        new() { Id = 4, Name = "Матча-латте", Description = "Зелений чай матча з молоком.",
BasePrice = 90m },
        new() { Id = 5, Name = "Какао", Description = "Шоколадний напій без кави.",
BasePrice = 60m },
        new() { Id = 6, Name = "Чай-латте", Description = "Пряний чай із молоком.",
BasePrice = 75m }
    };

    private void SelectDrink(Drink drink) => selectedDrink = drink;
    private void CloseDialog() => selectedDrink = null;
    private void ConfirmDrink(OrderLine line)
    {
        Order.Add(line);
        CloseDialog();
    }
}
```

Стало — повний вміст після заміни:

```
@page "/"
@inject CatalogService Catalog
@inject OrderState Order
@inject ILogger<Home> Logger

<PageTitle>Меню · Campus Café</PageTitle>

<section class="hero">
    <p class="eyebrow">LNTU · кава між парами</p>
    <h1>Campus Café</h1>
    <p>Оберіть напій, налаштуйте смак і зберіть своє замовлення.</p>
</section>

<div class="shop-grid">
    <section aria-label="Каталог напоїв">
        <div class="section-heading"><h2>Наше меню</h2><span>6 приводів зробити
паузу</span></div>
        @if (error is not null)
        {
            <p role="alert">@error</p>
            <button type="button" @onclick="LoadAsync">Спробувати ще</button>
        }
        else if (drinks is null)
        {
            <p role="status">Завантаження меню…</p>
```

```
        }
        else if (drinks.Count == 0)
        {
            <p>Меню поки що порожнє.</p>
        }
        else
        {
            <div class="catalog">
                @foreach (var drink in drinks)
                {
                    <DrinkCard @key="drink.Id" Drink="drink" OnSelect="SelectDrink" />
                }
            </div>
        }
    </section>
    <CartSummary />
</div>

@if (selectedDrink is not null)
{
    <ConfigureDrinkDialog @key="selectedDrink.Id" Drink="selectedDrink"
                          OnCancel="CloseDialog" OnConfirm="ConfirmDrink" />
}

@code {
    private List<Drink>? drinks;
    private Drink? selectedDrink;
    private string? error;

    protected override Task OnInitializedAsync() => LoadAsync();

    private async Task LoadAsync()
    {
        error = null;
        drinks = null;
        try
        {
            drinks = await Catalog.GetDrinksAsync();
        }
        catch (Exception exception)
        {
            Logger.LogError(exception, "Не вдалося завантажити каталог");
            error = "Не вдалося завантажити меню. Перевірте сервер і спробуйте ще раз.";
        }
    }

    private void SelectDrink(Drink drink) => selectedDrink = drink;
    private void CloseDialog() => selectedDrink = null;

    private void ConfirmDrink(OrderLine line)
    {
        Order.Add(line);
        CloseDialog();
    }
}
```

`OnInitializedAsync()` запускається під час ініціалізації компонента. Поки `drinks` дорівнює `null`, показується повідомлення про завантаження. Після `await` Blazor відображає отриманий список. Порожній список і помилка — окремі стани, не нескінченне «Завантаження».

`ILogger<Home>` записує технічні відомості на сервері. Користувачеві показується коротке повідомлення, а не текст винятку зі шляхами чи деталями бази даних.

`selectedDrink` спочатку `null`, тому діалогу немає. `SelectDrink` встановлює вибраний напій — діалог з'являється. `CloseDialog` повертає `null` — компонент прибирається з дерева разом із його чернеткою. Наступне відкриття створить нову чернетку зі значеннями за замовчуванням.

Повний шлях події:

```
Клік «Обрати»
    → DrinkCard.OnSelect
    → Home.SelectDrink
    → ConfigureDrinkDialog
    → @bind змінює DrinkDraft
    → OnConfirm передає OrderLine
    → Home.ConfirmDrink
    → OrderState.Add
    → Changed
    → CartSummary оновлює список і суму
```

Основний каталог викликає `CatalogService` безпосередньо, бо C# цього режиму вже виконується на сервері. JSON API використовує той самий сервіс. Немає потреби змушувати серверний компонент звертатися HTTP-запитом до самого себе; окрему вправу з `HttpClient` наведено далі.

Контроль: сторінка показує шість напоїв із SQLite; вибір відкриває діалог, підтвердження наповнює кошик, скасування не змінює його. Оформлення вже успадковане з попередньої роботи.

8. Спільний макет і каскадні налаштування

Звичайний параметр передається конкретному дочірньому компоненту. **Каскадний параметр** доступний компонентам нижче в дереві без повторення атрибута на кожному проміжному рівні.

Адаптуйте зараз — назва й тексти. У `ShopSettings.Name` задайте назву свого магазину. У `MainLayout.razor` змініть іконку, `aria-label`, підвал і написи навігації; слово «Меню» за потреби замініть на «Каталог». Перевірте також `PageTitle` в обох сторінках: назва вкладки не змінюється автоматично разом із `ShopSettings.Name`. Символ гривні та `MoneyLabel` можна залишити спільними.

Створіть **Models/ShopSettings.cs**:

```
namespace CampusCafe.Models;

public class ShopSettings
{
    public string Name { get; } = "Campus Café";
```

```
    public string CurrencySymbol { get; } = "₴";
}
```

Змініть **Components/Layout/MainLayout.razor** відносно результату лабораторної 7:

Було — вміст файла на попередньому етапі:

```
@inherits LayoutComponentBase

<header class="site-header">
    <a class="brand" href="/">☕ Campus Café</a>
    <nav aria-label="Основна навігація">
        <NavLink href="/" Match="NavLinkMatch.All">Каталог</NavLink>
    </nav>
</header>
<main class="page-shell">@Body</main>
<footer class="site-footer">Навчальний проєкт LNTU · Ціни у гривнях</footer>

<div id="blazor-error-ui" data-nosnippet>
    Роботу застосунку перервано.
    <a href="." class="reload">Перезавантажити сторінку</a>
    <span class="dismiss">×</span>
</div>
```

Стало — повний вміст після заміни:

```
@inherits LayoutComponentBase

<CascadingValue Value="settings" IsFixed="true">
    <header class="site-header">
        <a class="brand" href="/" aria-label="Campus Café — головна"><span aria-
hidden="true">☕</span> @settings.Name</a>
        <nav aria-label="Основна навігація">
            <NavLink href="/" Match="NavLinkMatch.All">Меню</NavLink>
            <NavLink href="/about">Про кав’ярню</NavLink>
        </nav>
    </header>
    <main class="page-shell">@Body</main>
    <footer class="site-footer">Campus Café · Навчальний проєкт LNTU · Ціни у
гривнях</footer>
</CascadingValue>

<div id="blazor-error-ui" data-nosnippet>
    З’єднання або роботу застосунку перервано.
    <a href="." class="reload">Перезавантажити сторінку</a>
    <span class="dismiss">×</span>
</div>

@code {
    private readonly ShopSettings settings = new();
}
```

`@Body` — місце для вмісту поточної сторінки. `CascadingValue` передає об’єкт `settings` нащадкам. `IsFixed="true"` підходить, бо об’єкт налаштувань не змінюється протягом життя макета. Це не

спосіб повідомляти про зміни кошика.

Залиште наявний **MainLayout.razor.css**: він оформлює повідомлення про помилку Blazor. Файли `ReconnectModal.*` також залиште з шаблону. Не задавайте окремий `@rendermode` на макеті чи кожному дочірньому компоненті — вони вже в інтерактивному дереві `Routes`.

Components/Shared/MoneyLabel.razor уже існує. Замініть просте форматування версією з каскадним параметром.

Було:

```
@using System.Globalization
<span>@Amount.ToString("N2", CultureInfo.GetCultureInfo("uk-UA")) ₴</span>

@code {
    [Parameter] public decimal Amount { get; set; }
}
```

Стало:

```
@using System.Globalization
<span>@Amount.ToString("N2", CultureInfo.GetCultureInfo("uk-UA"))
@Shop.CurrencySymbol</span>

@code {
    [Parameter] public decimal Amount { get; set; }
    [CascadingParameter] public ShopSettings Shop { get; set; } = default!;
}
```

`[Parameter] Amount` надходить від безпосереднього батька, а `[CascadingParameter] Shop` — із макета. `default!` пригнічує попередження компілятора до встановлення параметра; він не створює об’єкт. Цей компонент слід використовувати всередині нашого `CascadingValue`.

`N2` показує два десяткові знаки, а `uk-UA` задає українське числове форматування. Символ `₴` додається явно один раз. CSS не вставляє жодних валютних символів. Ціни в базі вже задано в гривнях; форматування не здійснює валютного обміну.

Імпорти моделей, спільних компонентів і сервісів уже є. Повторно додавати їх не потрібно. Посилання «Про кав’ярню» запрацює після наступного кроку.

9. Друга сторінка та тривалість життя стану

Адаптуйте зараз — друга сторінка. Опишіть власний магазин, змініть назву вкладки та слово «кав’ярня» в навігації. Залиште пояснення поведінки кошика, якщо у вас він так само зберігається лише в пам’яті сесії.

Створіть **Components/Pages/About.razor**:

```
@page "/about"
<PageTitle>Про кав’ярню · Campus Café</PageTitle>
<section class="hero">
    <p class="eyebrow">Про проєкт</p>
    <h1>Ваша кава.<br /><em>Ваші правила.</em></h1>
    <p>Каталог читається з SQLite, а кошик зберігається в пам’яті поточної інтерактивної
сесії.</p>
    <p>Поверніться в меню через навігацію — кошик залишиться. Після повного
перезавантаження сторінки створюється нова сесія.</p>
    <a class="button-link" href="/">Повернутися до меню</a>
</section>
```

Маршрут `/about` визначається директивою `@page`, а не лише ім’ям файла. У **Components/Routes.razor** залиште маршрутизатор із шаблону:

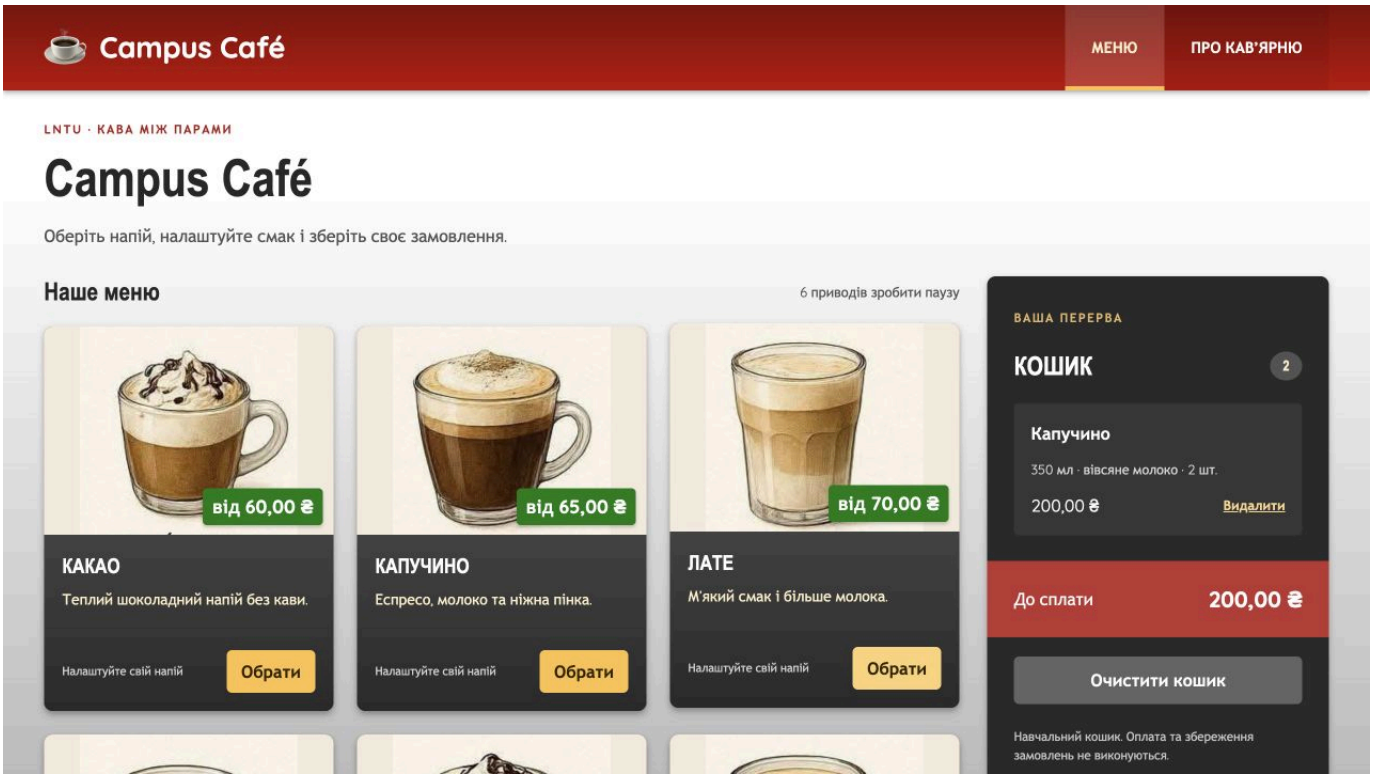
```
<Router AppAssembly="typeof(Program).Assembly" NotFoundPage="typeof(Pages.NotFound)">
    <Found Context="routeData">
        <RouteView RouteData="routeData" DefaultLayout="typeof(Layout.MainLayout)" />
        <FocusOnNavigate RouteData="routeData" Selector="h1" />
    </Found>
</Router>
```

Додайте напій до кошика. Перейдіть посиланням «Про кав’ярню», потім «Меню». У глобальному інтерактивному дереві це навігація в межах сесії: `Home` і `CartSummary` створюються знову, але отримують той самий `OrderState`.

Тепер повністю перезавантажте браузер. У цьому демо кошик стане порожнім, а каталог залишиться: перший був у пам’яті сесії, другий — у файлі SQLite. Відкрийте окрему вкладку: це також окрема інтерактивна сесія з власним кошиком.

10. Оформлення й остаточна структура

CSS, фотографії та шрифти залишаються з лабораторної 7. Стилі `.shop-grid` і `.cart`, які раніше не використовувалися, тепер оформлюють каталог із кошиком. Не замінюйте власне оформлення демонстраційним повторно.




```
CampusCafe/
├── Components/
│   ├── App.razor
│   ├── Routes.razor
│   ├── _Imports.razor
│   ├── Layout/
│   │   ├── MainLayout.razor
│   │   ├── MainLayout.razor.css
│   │   └── ReconnectModal.razor (+ .css та .js)
│   ├── Pages/
│   │   ├── Home.razor
│   │   ├── About.razor
│   │   ├── Error.razor
│   │   └── NotFound.razor
│   └── Shared/
│       ├── MoneyLabel.razor
│       ├── DrinkCard.razor
│       ├── ConfigureDrinkDialog.razor
│       └── CartSummary.razor
├── Data/
│   ├── CafeDbContext.cs
│   └── SeedData.cs
├── Models/
│   ├── Drink.cs
│   ├── DrinkDraft.cs
│   ├── OrderLine.cs
│   └── ShopSettings.cs
├── Services/
│   ├── CatalogService.cs
│   └── OrderState.cs
├── Properties/launchSettings.json
├── wwwroot/
│   ├── app.css
│   ├── images/coffee-cups.jpg
```

```
│   └── fonts/
│       ├── quicksand-v8-latin-regular.woff2
│       ├── quicksand-v8-latin-700.woff2
│       └── OFL-Quicksand.txt
├── appsettings.json
├── appsettings.Development.json
├── CampusCafe.csproj
├── Program.cs
└── cafe.db                          створюється при запуску
```

`bin` та `obj` створюються під час збирання. Їх не потрібно переносити як вихідний код. `cafe.db` також не обов'язково роздавати для чистого початку роботи: початковий каталог створиться автоматично.

11. Перевірка результату

Спочатку перевірте компіляцію:

```
dotnet build
dotnet run --urls http://localhost:5127
```

Перевірте власну тематику. Таблиця нижче містить контрольні числа демонстрації Campus Café. У своєму застосунку замініть їх власними товарами й заздалегідь розрахованими сумами. Обов'язково перевірте базову конфігурацію, дві платні опції, кількість понад одну одиницю та видалення однієї з кількох позицій.

Виконайте такі сценарії:

Дія	Очікуваний результат
Відкрити каталог	Шість карток із цінами у гривнях.
Капучино, 250 мл, звичайне молоко, 1 шт.	65,00 ₴.
Капучино, 350 мл, вівсяне молоко, 2 шт.	100,00 ₴ за один; 200,00 ₴ за позицію.
Змінити опції та скасувати	Кошик не змінюється.
Знову відкрити напій	Нова чернетка: 250 мл, звичайне молоко, 1 шт.
Додати ще базове лате	Два рядки, три порції, загалом 270,00 ₴.
Видалити капучино	Залишається лате й 70,00 ₴.
Очистити кошик	Порожній стан, кількість 0.
Перейти через меню на `/about` і назад	Кошик зберігається.
Повністю перезавантажити сторінку	Кошик порожній; каталог зберігається.
Відкрити `/api/drinks`	JSON із шістьма напоями.
Відкрити невідому адресу	Сторінка 404 шаблону.
Звузити вікно до розміру телефона	Каталог і кошик розташовуються послідовно.

У комплекті демопроєкту є **CampusCafe.Checks** — консольна перевірка формул, обмежень, кошика й SQLite. Із кореня комплекту виконайте:

```
dotnet run --project CampusCafe.Checks
```

Ці перевірки виконують C#-логіку без браузера й не замінюють перевірку інтерфейсу. `CampusCafe.Checks` перевіряє саме демонстрацію: кавові назви та числа не є очікуваними значеннями вашого проєкту. Якщо переносите цей проєкт перевірок, змініть посилання на `.csproj`, простори імен, типи та контрольні ціни. Навіть без автоматичних тестів підготуйте власну таблицю «вхідні параметри → очікувана сума → фактична сума».

12. Типові труднощі

Симптом	Що перевірити
Кнопки видно, але вони не працюють	Реєстрації Interactive Server у `Program.cs`, режим `Routes` в `App.razor`, завантаження `blazor.web.js`, з'єднання із сервером.
Не знайдено `UseSqlite`	Пакет EF Core SQLite 10 та `using Microsoft.EntityFrameworkCore`.
Не знайдено `DrinkCard` або `MoneyLabel`	Назви файлів, папку `Components/Shared` і імпорт її простору імен.
Помилка отримання `OrderState`	`AddScoped<OrderState>()` до `builder.Build()`.
`MoneyLabel` отримує порожній `Shop`	Розміщення всередині `CascadingValue` у макеті.
Зміни seed не видно в каталозі	Існуюча база вже містить записи; seed не перезаписує їх.
Не змінився кошик	Не створюйте власний `new OrderState`; перевірте `Changed` і підписку компонента.
Після F5 зник кошик	Це очікувано для стану в пам'яті сесії без постійного зберігання.
Ціна має крапку чи неправильний символ	Виводьте її через `MoneyLabel`, не додавайте валюту ще раз у CSS.
API повертає HTML замість JSON	Перевірте адресу `/api/drinks`, запущений порт і наявність `MapGet`.
SQLite повідомляє про відсутній стовпець	Схему моделі змінено після створення бази; потрібна міграція або нова окрема навчальна база.

13. Підсумок: механізми, файли та подальше використання

13.1. Три різні види даних

У застосунку є не один «товар», а три об'єкти з різними завданнями.

Об'єкт	На яке запитання відповідає	Коли існує
`Drink` — товар каталогу	Що продаємо і яка базова ціна?	Зберігається у SQLite та завантажується для показу.
`DrinkDraft` — чернетка вибору	Які опції користувач обирає зараз?	Поки відкритий компонент налаштування.
`OrderLine` — позиція кошика	Що користувач уже підтвердив і скільки це коштує?	У контейнері стану поточної сесії.

Це розділення корисне і для букетів, і для друку, і для замовлення одягу. Редагування чернетки не повинно змінювати товар у каталозі чи раніше підтверджену позицію. Скасування просто відкидає чернетку. Підтвердження створює **новий** об'єкт позиції.

У `OrderLine` збережено ціну одиниці на момент додавання. `Total` обчислюється з неї та кількості. У повноцінному магазині перед остаточним оформленням сервер ще раз перевіряє актуальні ціни й доступність товару; стан навчального кошика сам по собі не є оформленим замовленням.

13.2. Один вибір користувача — весь шлях даних

```
Відкриття сторінки
  Home.OnInitializedAsync
    → CatalogService
      → новий CafeDbContext через фабрику
        → запит до SQLite
      ← список товарів
    → картки отримують товар через [Parameter]

Вибір і налаштування
  кнопка картки → EventCallback<Drink> → Home
    → Home показує діалог для вибраного товару
      → @bind змінює чернетку
      → формула обчислює попередню ціну

Підтвердження
  діалог → EventCallback<OrderLine> → Home
    → OrderState.Add
      → подія Changed
        → CartSummary викликає StateHasChanged через InvokeAsync
          → новий список, кількість і сума в інтерфейсі
```

Картка не знає, як зберігається кошик. Діалог не виконує SQL-запити. Контейнер кошика не вирішує, скільки коштує вівсяне молоко чи подарункове пакування. Кожна частина відповідає за своє завдання — тому їх легше змінювати окремо.

13.3. Основні механізми — коротка пам’ятка

Механізм	Що він робить у цій роботі	На що звернути увагу
Razor-компонент `.razor`	Поєднує розмітку й C#-логіку частини інтерфейсу.	Компонент із `@page` доступний за адресою; без `@page` його включає інший компонент.
Interactive Server	Виконує обробники C# на сервері й передає зміни інтерфейсу браузеру.	Втрата з’єднання впливає на інтерактивність. Це не виконання всього C# у браузері.
`@if`, `@foreach`, `@key`	Показують потрібний стан, повторюють картки та зберігають відповідність елементів об’єктам.	Для ключа позиції використовуйте її `Id`, а не назву товару, що може повторюватися.
`[Parameter]`	Передає дані від батька конкретному дочірньому компоненту.	Переданий товар каталогу не використовуйте як змінювану чернетку замовлення.
`EventCallback<T>`	Повідомляє батька про вибір або підтвердження й передає значення.	Тип `T` описує дані події; повертайте `Task`, коли обробник асинхронний.
`@bind`	Показує значення властивості в елементі й оновлює її після введення.	`oninput` та `onchange` спрацьовують у різні моменти. Зміна C#-властивості не записує її автоматично в базу.
`CascadingValue` / `[CascadingParameter]`	Передає спільні налаштування магазину нащадкам.	Це спосіб доставити значення по дереву компонентів, а не автоматичне постійне сховище.
DI: `AddScoped` / `@inject`	Реєструє сервіс і надає його компонентам.	У нашому інтерактивному дереві scoped-кошик спільний для однієї сесії. `new OrderState()` у компоненті створив би інший кошик.
C#-подія `Changed`	Сповіщає компоненти-підписники про зміну сервісу стану.	Після підписки потрібна відписка в `Dispose`; сама зміна звичайного списку не повідомляє довільні компоненти.
`OnInitializedAsync` / `await`	Асинхронно завантажує каталог під час створення компонента.	Розрізняйте завантаження, порожній результат і помилку. Не записуйте замовлення як побічний ефект ініціалізації.

Механізм	Що він робить у цій роботі	На що звернути увагу
EF Core і `IDbContextFactory`	Створюють короткоживучий контекст для окремої операції з SQLite.	Не зберігайте один контекст для всіх користувачів. Звільняйте його через `await using`.
Minimal API / `HttpClient`	Надають каталог через HTTP і дозволяють його отримати як JSON.	API — окремий шлях доступу. Основний серверний компонент може викликати сервіс без HTTP.

Після звичайного обробника Blazor, наприклад `@onclick`, компонент зазвичай рендериться повторно автоматично. Повідомлення від зовнішнього для компонента сервісу — інша ситуація: для цього в кошику використано підписку й `InvokeAsync(StateHasChanged)`.

13.4. Карта файлів

Структуру зручно читати за відповідальністю, а не за порядком створення:

```
CampusCafe/
├── Program.cs                запуск, DI, маршрути, підготовка бази
├── Models/                   що описують дані та як рахується ціна
│   ├── Drink.cs
│   ├── DrinkDraft.cs
│   ├── OrderLine.cs
│   └── ShopSettings.cs
├── Data/                     робота з базою та початковим каталогом
│   ├── CafeDbContext.cs
│   └── SeedData.cs
├── Services/                 отримання каталогу й поточний стан
│   ├── CatalogService.cs
│   └── OrderState.cs
├── Components/
│   ├── App.razor             HTML-оболонка та режим інтерактивності
│   ├── Routes.razor          вибір сторінки за адресою
│   ├── _Imports.razor        спільні імпорти для компонентів
│   ├── Layout/               макет і компоненти шаблону
│   │   ├── MainLayout.razor
│   │   └── ReconnectModal.razor (+ .css та .js)
│   ├── Pages/                маршрутизовані сторінки
│   │   ├── Home.razor
│   │   ├── About.razor
│   │   ├── Error.razor
│   │   └── NotFound.razor
│   └── Shared/               повторно використовувані елементи
│       ├── DrinkCard.razor
│       ├── ConfigureDrinkDialog.razor
│       ├── CartSummary.razor
│       └── MoneyLabel.razor
└── wwwroot/                  CSS, локальні зображення та шрифти
```

Це карта основних файлів; повний перелік конфігураційних і допоміжних файлів наведено в розділі 10. Папки самі по собі не підключають сервіси: наявність `OrderState.cs` у `Services` не замінює реєстрацію в `Program.cs`.

Разом у двох роботах створено 13 файлів застосунку:

Новий файл	Для чого створений
`Models/Drink.cs`	Опис товару каталогу, що зберігається в базі.
`Models/DrinkDraft.cs`	Поточні опції, перевірка допустимих значень і розрахунок ціни.
`Models/OrderLine.cs`	Знімок підтвердженої позиції: товар, опції, кількість і ціна.
`Models/ShopSettings.cs`	Спільна назва магазину та символ валюти.
`Data/CafeDbContext.cs`	Налаштований тип EF Core-контексту й набір товарів.
`Data/SeedData.cs`	Створення й початкове наповнення навчальної бази.
`Services/CatalogService.cs`	Читання каталогу без SQL-логіки всередині UI-компонентів.
`Services/OrderState.cs`	Список позицій, операції кошика й повідомлення про зміни.
`Components/Shared/DrinkCard.razor`	Відображення товару й повідомлення про його вибір.
`Components/Shared/ConfigureDrinkDialog.razor`	Введення опцій, попередня сума, підтвердження або скасування.
`Components/Shared/CartSummary.razor`	Відображення стану кошика та дії над його позиціями.
`Components/Shared/MoneyLabel.razor`	Єдине форматування всіх грошових сум.
`Components/Pages/About.razor`	Друга сторінка для власного опису й перевірки навігації.

У власній темі назви можуть відрізнятися, але ці відповідальності залишаються.

Змінено у файлах, які вже створив шаблон:

Файл	Що змінилося
`Program.cs`	Додано фабрику контекстів, сервіси, GET API й виклик seed.
`CampusCafe.csproj`	Команда встановлення пакета додала залежність EF Core SQLite.
`Components/App.razor`	Українська мова документа, Interactive Server без попереднього рендерингу.
`Components/_Imports.razor`	Імпорти моделей, сервісів і спільних компонентів.
`Components/Layout/MainLayout.razor`	Шапка, навігація, підвал і каскадне значення налаштувань.
`Components/Pages/Home.razor`	Завантаження каталогу, показ карток, відкриття діалогу та обробка результату.
`wwwroot/app.css`	Оформлення й адаптивний макет.

`Routes.razor`, сторінки помилок і `ReconnectModal` у базовому варіанті залишаються зі шаблону. `cafe.db` не створюють як текстовий файл: його готує EF Core під час запуску. Зображення й шрифти — ресурси, а не моделі чи сервіси. `CampusCafe.Checks` у готовому комплекті — окремий допоміжний проєкт перевірок, не частина дерева UI.

13.5. Що зберігається, а що зникає

Дія	Каталог у SQLite	Чернетка	Кошик цієї версії
Скасування діалогу	Без змін	Відкидається	Без змін
Підтвердження діалогу	Без змін	Компонент закривається	Додається нова позиція
Перехід на іншу інтерактивну сторінку	Зберігається	Компонент сторінки прибирається	Зберігається в тій самій сесії
Повне перезавантаження браузера	Зберігається	Створюється заново при відкритті діалогу	Нова сесія має порожній кошик
Перезапуск сервера	Зберігається у файлі	Втрачається	Стан у пам’яті втрачається

Ця поведінка відповідає саме реалізованому демо: у ньому немає збереження кошика в базі, браузерному сховищі чи механізму його відновлення після нового сеансу.

13.6. План для наступного проєкту

1. **Визначте сутності.** Відокремте товар каталогу, змінювану чернетку та підтверджену позицію. Запишіть одиниці вимірювання й формулу до створення форми.

2. **Перевірте правила без UI.** Для кількох наборів параметрів обчисліть очікувані суми; перевірте нульову, надмірну й неприпустиму кількість.
3. **Підключіть дані.** Створіть модель, контекст, початковий каталог і сервіс читання. Вирішіть, чи клієнту потрібен HTTP API, чи достатньо прямого серверного виклику.
4. **Налаштуйте запуск.** Зареєструйте залежності до `builder.Build()`, маршрути — після нього; перевірте обраний режим рендерингу.
5. **Розділіть UI.** Картка показує дані, діалог збирає опції, батьківська сторінка координує дії, кошик відображає стан.
6. **Продумайте тривалість життя даних.** Що достатньо тримати в компоненті, що спільне для сесії, а що має зберігатися після перезапуску?
7. **Перевірте переходи й помилки.** Скасування, повторне відкриття, однакові товари з різними опціями, порожній результат, втрата даних і помилка запиту важливі так само, як успішне додавання.

Можна повторно використати підхід до компонентів, DI, контейнера стану, фабрики контекстів і форматування валюти. Дані seed, опції чернетки, формулу, тематичні підписи та зображення слід спроєктувати для нової задачі.

Для публічного сервісу до цієї основи окремо додають HTTPS, необхідні правила доступу, валідацію операцій на сервері, міграції схеми, постійне зберігання замовлень і повноцінну клавіатурну доступність діалогів. Навчальний приклад демонструє механізми Blazor, але не замінює ці вимоги.

Нове саме в лабораторній 8

Сім нових файлів: `Services/OrderState.cs`, `Services/CatalogService.cs`, `Data/CafeDbContext.cs`, `Data/SeedData.cs`, `Models/ShopSettings.cs`, `Components/Shared/CartSummary.razor`, `Components/Pages/About.razor`.

Змінені файли: `Program.cs` — реєстрації й запуск seed; `.csproj` — пакет SQLite; `_Imports.razor` — сервіси; `Home.razor` — кошик і завантаження; `ConfigureDrinkDialog.razor` — напис кнопки; `MainLayout.razor` та `MoneyLabel.razor` — каскадні налаштування. Моделі товару, чернетки й позиції та компонент картки використовуються повторно.

На захисті покажіть шлях `Home → CatalogService → DbContext → SQLite` і окремо `OnConfirm → OrderState.Add → Changed → CartSummary`. Поясніть, чому ці два шляхи не є однією операцією збереження замовлення.

14. Запитання для самоперевірки

1. Де виконується `GetUnitPrice`: у браузері чи на сервері?
2. Чому картка отримує `Drink`, а кошик містить `OrderLine`?
3. Чим `@bind` відрізняється від простого виведення `@draft.Volume`?
4. Як `EventCallback<OrderLine>` повертає результат діалогу батьківській сторінці?
5. Чому `Changed` — не те саме, що `@onclick`?
6. Для чого потрібна відписка в `Dispose()`?

7. Чому контекст бази створюється на одну операцію, а кошик живе протягом сесії?
8. Які дані переживуть перезапуск сервера?
9. Що потрібно змінити, щоб каталог отримував справжній клієнт WebAssembly?
10. Чому грошова сума рахується через `decimal`, а форматування не змінює валюту?

15. Необов'язковий дослід: навіщо каталогу HTTP API

Яке запитання досліджуємо

У готовому основному варіанті каталог уже працює. Компонент `Home` викликає C#-метод `Catalog.GetDrinksAsync()`, а той читає SQLite. Для цього **не потрібен HTTP-запит до `/api/drinks`**. Маршрут API, створений у розділі 6, існує паралельно, але сторінка ним ще не користується.

Уявіть, що ці самі товари має показувати ще й мобільний застосунок. Він не може просто викликати метод сервісу всередині процесу нашого сервера. Йому потрібна доступна через мережу адреса, яка повертає дані у погодженому форматі. Такою адресою є `/api/drinks`, а форматом — JSON.

Мета досліду: отримати той самий каталог іншим шляхом і розрізнити прямий виклик C#-методу та мережевий запит. Вигляд карток, ціни й робота кошика після зміни мають залишитися такими самими.

```
Основний варіант:
Home → CatalogService → SQLite → List<Drink>

Варіант досліду:
Home → HttpClient → GET /api/drinks → CatalogService → SQLite
Home ← List<Drink> ← перетворення JSON ← HTTP-відповідь ← дані
```

У першому випадку між `Home` і сервісом немає HTTP, коду статусу або JSON. У другому з'являються URL, GET-запит, HTTP-відповідь і перетворення JSON назад на C#-об'єкти.

Для Interactive Server додатковий запит до власного API не є необхідним покращенням: він додає обмін даними й можливі помилки. Його використовуємо тут для порівняння. Для зовнішнього клієнта така межа доступу до даних має практичний сенс.

Перед початком

Виконуйте дослід лише після того, як основна версія працює. Збережіть її окремим комітом або копією **свого** проєкту. Не змінюйте моделі, seed чи формулу ціни. Якщо у вас інша назва моделі або маршруту API, використовуйте свої відповідники замість `Drink` і `/api/drinks`.

Відкрийте `http://localhost:5127/api/drinks` (або власний маршрут на вашому порту) й переконайтеся, що бачите JSON свого каталогу. Це перевіряє API, але ще не означає, що сторінка вже отримує дані через нього.

Зміна способу завантаження

Виконайте три локальні зміни:

1. У **Program.cs**, до `builder.Build()`, додайте:

```
--- a/Program.cs
+++ b/Program.cs
@@ -13,6 +13,7 @@
     options.UseSqlite($"Data Source={databasePath}"));
 builder.Services.AddScoped<CatalogService>();
 builder.Services.AddScoped<OrderState>();
+builder.Services.AddHttpClient();

 var app = builder.Build();
```

2. У **Home.razor** замість `@inject CatalogService Catalog` додайте:

```
--- a/Components/Pages/Home.razor
+++ b/Components/Pages/Home.razor
@@ -1,5 +1,6 @@
 @page "/"
-@inject CatalogService Catalog
+@inject HttpClient Http
+@inject NavigationManager Navigation
 @inject OrderState Order
 @inject ILogger<Home> Logger
```

3. У `LoadAsync()` замініть тільки рядок отримання списку:

```
--- a/Components/Pages/Home.razor
+++ b/Components/Pages/Home.razor
@@ -60,7 +60,8 @@
         drinks = null;
         try
         {
-            drinks = await Catalog.GetDrinksAsync();
+            drinks = await Http.GetFromJsonAsync<List<Drink>>(
+                Navigation.BaseUri + "api/drinks") ?? new List<Drink>();
         }
         catch (Exception exception)
         {
```

`HttpClient` надсилає HTTP-запити. `GetFromJsonAsync<List<Drink>>()` виконує GET, читає JSON і перетворює його на список C#-об'єктів. Тут це заміна **одного способу отримання даних іншим**, а не перенесення всього застосунку в браузер.

`System.Net.Http.Json` уже імпортовано в `_Imports.razor`. `Navigation.BaseUri` містить фактичну базову адресу, тому не потрібно жорстко записувати порт у компоненті.

Шлях даних тепер такий: компонент → HTTP GET → Minimal API → сервіс → SQLite → JSON → список `Drink`. У Interactive Server цей HTTP-запит виконує **сервер**, а не JavaScript у браузері. Тому його не обов'язково буде видно як окремий запит каталогу в мережевій панелі браузера.

Як переконатися, що шлях даних змінився

1. Перезапустіть застосунок. На головній сторінці мають бути ті самі власні товари; повторіть один розрахунок і перевірте кошик.
2. Поставте точку зупину в обробнику `MapGet` у `Program.cs` і перезавантажте головну сторінку. У HTTP-варіанті її завантаження повинно привести до цього обробника. У початковому варіанті просте завантаження каталогу його не викликало.
3. Для короткого досліду змініть **лише адресу в** `GetFromJsonAsync` на `api/no-such-catalog` і перезапустіть застосунок. Очікуйте повідомлення «Не вдалося завантажити меню...»: HTTP-запит завершується помилкою, а її обробляє `catch` у `LoadAsync`. Поверніть правильний маршрут і перевірте відновлення каталогу. Не змінюйте при цьому `MapGet` чи базу.
4. Поясніть, чому хибний URL не впливав би на прямий виклик `Catalog.GetDrinksAsync()`.

Яку версію залишити

Для основного завдання достатньо прямого виклику сервісу. Поверніть його після досліду або залиште HTTP-варіант як окрему копію для порівняння. Сам по собі цей дослід не додає автентифікацію, захист API чи підтримку WebAssembly.

Порівняйте обидва варіанти. Щоб повернути основну версію, застосуйте три diff цього розділу у зворотному напрямку: вилучіть рядки з `+` і поверніть рядки з `-`. Це відновить і виклик `Catalog.GetDrinksAsync()`, і його `@inject CatalogService Catalog`; реєстрація `AddHttpClient()` більше не знадобиться. Для клієнта WebAssembly чи мобільного застосунку API є природною межею доступу до серверних даних. Зміна лише `@rendermode` на WebAssembly не перенесе SQLite-сервіс у браузер.

Джерела для поглибленого вивчення

- Режими рендерингу Blazor.
- Використання EF Core у серверному Blazor.
- Прив'язування даних у компонентах.
- Обробка подій і EventCallback.
- Каскадні значення та параметри.

Джерела оформлення та ресурсів: Blazing Pizza, ілюстрація чашок. Відомості про шрифти й походження ілюстрації містяться в `ASSETS.md` комплекту демопроєкту.